

Teaching Exam

**Subject –
Computer
Science**

**Operating
System**



Lecture No –6

By–Satyendra Chaudhary Sir

Topics to be Covered

Topic

Process synchronization.



Process Synchronization

- Concurrent access to shared data may result in data inconsistency.
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.
- Shared-memory solution to bounded-buffer problem (Chapter 4) allows at most $n - 1$ items in buffer at the same time. A solution, where all N buffers are used is not simple.
 - Suppose that we modify the producer-consumer code by adding a variable *counter*, initialized to 0 and incremented each time a new item is added to the buffer

Q The following two functions P_1 and P_2 that share a variable B with an initial value of 2 execute concurrently.

$P_1()$	$P_2()$
{	{
$C = B - 1;$	$D = 2 * B;$
$B = 2 * C;$	$B = D - 1;$
}	}

The number of distinct values that B can possibly take

Race Condition

- **Race condition:** The situation where several processes access – and manipulate shared data concurrently. The final value of the shared data depends upon which process finishes last.
- To prevent race conditions, concurrent processes must be **synchronized**.

General Structure of a process }

✓ Initial Section: Where process is accessing private resources.

• Entry Section: Entry Section is that part of code where, each process request for permission to enter its critical section.

• Critical Section: Where process is access shared resources.

• Exit Section: It is the section where a process will exit from its critical section.

• Remainder Section: Remaining Code.

Process execute

critical $\rightarrow$ code of process

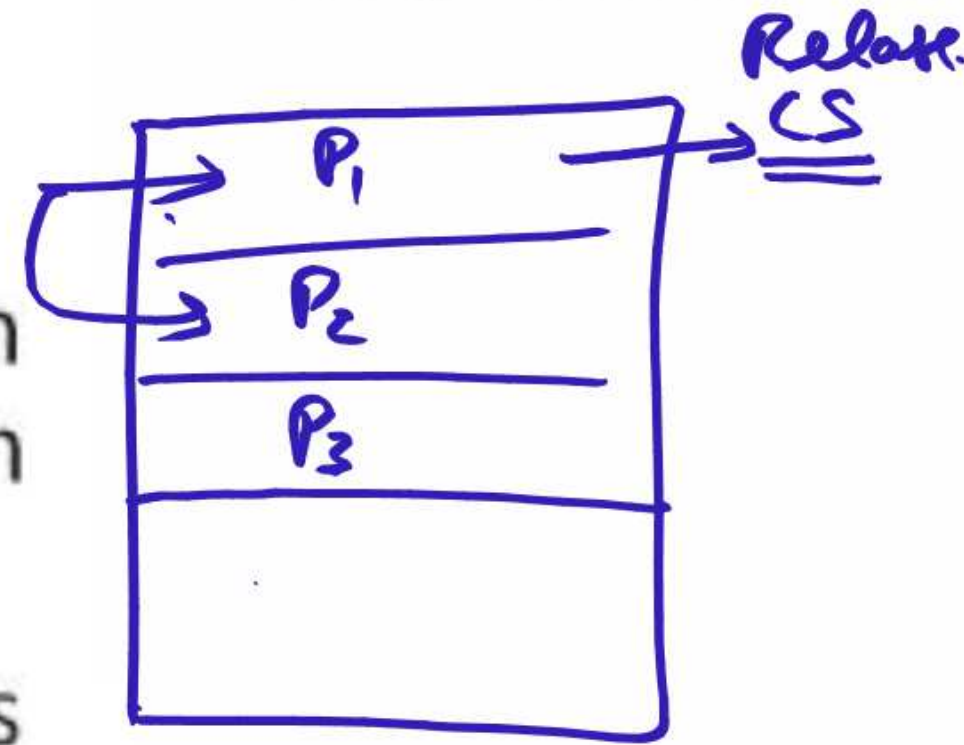
The Critical-Section Problem

- n processes all competing to use some shared data
- Each process has a code segment, called *critical section*, in which the shared data is accessed.
- Problem – ensure that when one process is executing in its critical section, no other process is allowed to execute in its critical section.

Solution to Critical-Section Problem

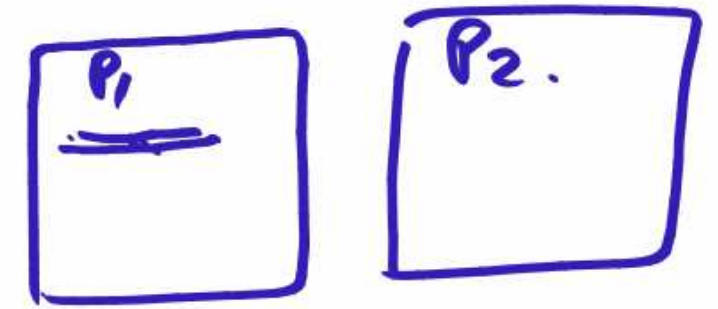
1. Mutual Exclusion. If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.

2. Progress. If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely.



Bounded Waiting.

A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.



- Assume that each process executes at a nonzero speed.
- No assumption concerning relative speed of the n processes.

We generally have the following solutions to a Critical Section Problems:

1. Two Process Solution //

1. Using Boolean variable turn
2. Using Boolean array flag
3. Peterson's Solution

2. Operating System Solution *n processes* //

1. Counting Semaphore —
2. Binary Semaphore —

3. Hardware Solution //

1. Test and Set Lock
2. Disable interrupt

Boolean Variable Turn. (Two process solⁿ)

P_0	P_1
<pre>while (1) { while (turn != 0); Critical Section turn = 1; Remainder section }</pre>	<pre>while (1) { while (turn != 1); Critical Section turn = 0; Remainder Section }</pre>

P_0

```

while (1)
{
    flag[0] = T;
    while (flag[1]);
    Critical Section
    flag[0] = F;
    Remainder section
}

```

Handwritten annotations for P_0 : A blue circle around the `while (1)` loop. A blue circle around `flag[1]` in the `while (flag[1]);` line. A blue arrow pointing from the `while (flag[1]);` line down to the `Remainder section` label.

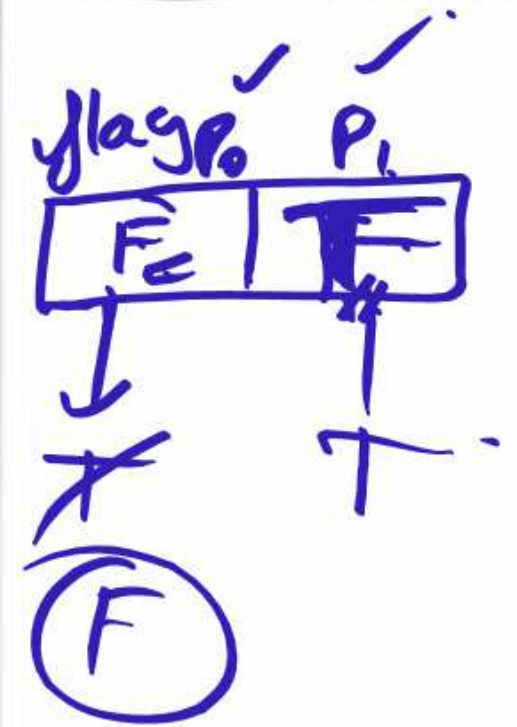
P_1

```

while (1)
{
    flag[1] = T;
    while (flag[0]);
    Critical Section
    flag[1] = F;
    Remainder Section
}

```

Handwritten annotations for P_1 : A blue circle around the `while (1)` loop. A blue circle around `flag[0]` in the `while (flag[0]);` line. A blue arrow pointing from the `while (flag[0]);` line down to the `Remainder Section` label.



Mutual Exclusion, Progress, Unbounded waiting.

P ₀	P ₁
<pre> while (1) { flag[0] = T; turn = 1; while (turn == 1 && flag[1] == T); Critical Section flag[0] = F; Remainder section } </pre>	<pre> while (1) { flag[1] = T; turn = 0; while (turn == 0 && flag[0] == T); Critical Section flag[1] = F; Remainder Section } </pre>

flag

F	T
--------------	---

turn = 0

T

Concurrent access to shared data may result in _____

- a) data consistency
- b) data insecurity
- c) data inconsistency ✓✓
- d) none of the mentioned

View Answer

Semaphore

n-process

variable
integer

A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which access takes place is called _____

- a) data consistency
- b) race condition //
- c) aging
- d) starvation

The segment of code in which the process may change common variables, update tables, write into files is known as _____

- a) program
- ☒ b) critical section
- c) non – critical section
- d) synchronizing

Which of the following conditions must be satisfied to solve the critical section problem?

- a) Mutual Exclusion
- b) Progress
- c) Bounded Waiting
- d) All of the mentioned

Mutual exclusion implies that _____

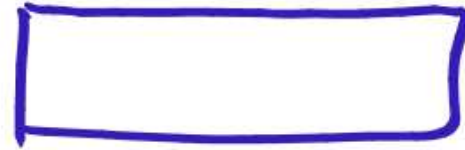
- a) if a process is executing in its critical section, then no other process must be executing in their critical sections ✓
- b) if a process is executing in its critical section, then other processes must be executing in their critical sections
- c) if a process is executing in its critical section, then all the resources of the system must be blocked until it finishes execution
- d) none of the mentioned

. Bounded waiting implies that there exists a bound on the number of times a process is allowed to enter its critical section _____

- a) ☒ after a process has made a request to enter its critical section and before the request is granted
- b) when another process is in its critical section
- c) before a process has made a request to enter its critical section
- d) none of the mentioned

A minimum of _____ variable(s) is/are required to be shared between processes to solve the critical section problem.

- a) one
- b) two ✓✓
- c) three
- d) four



Process synchronization can be done on _____

- a) hardware level
- b) software level
- c) both hardware and software level
- d) none of the mentioned

What will happen when a process terminates?

- a) It is removed from all queues ✓✓✓
- b) It is removed from all, but the job queue
- c) Its process control block is de-allocated
- d) Its process control block is never de-allocated



2 mins Summary



TEACHING
WALLAH

Topic

141.

process

pitelson solution

THANK YOU